From Blind Search to Memory-Aware Evolution: Efficient DBMS Tuning via Collaborative Diagnosis and Utility-Aware Retrieval

Anonymous Author(s)

Abstract

Modern DBMSs expose multiple configurable components (e.g., knobs, query hints, and indexes) that jointly determine query performance. Multi-component tuning is challenging due to the large combinatorial search space and the difficulty of learning effective tuning policies under limited feedback. Existing approaches still rely on blind search over the configuration space and interaction-heavy policy learning, leading to high tuning overhead and limited performance gains. Recent advances in large language models (LLMs) enable knowledge-driven tuning, but existing LLM-based methods fail to effectively exploit online feedback and historical observations, often converging prematurely to suboptimal configurations.

In this paper, we present EvoTune, a memory-aware evolution framework for multi-component DBMS tuning. EvoTune first localizes a query-specific high-impact subspace via collaborative diagnosis, which combines lightweight pattern learning with LLM-based reasoning. It further introduces a utility-aware retrieval policy that selects informative observations based on their resulting long-term performance improvement, instead of similarity-based retrieval. To support continual improvement, EvoTune organizes tuning feedback into a hierarchical memory and incrementally refines both subspace localization and tuning policies without requiring LLM fine-tuning. Extensive experiments show that EvoTune consistently outperforms state-of-the-art baselines, achieving up to 44.5% performance improvement under the same tuning budget and reaching the best competing baseline's final performance up to 3.9× faster.

Keywords

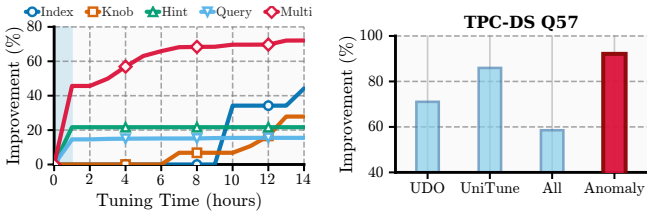
Multi-Component Tuning, Diagnosis, LLM-empowered, Evolution

1 Introduction

Query performance is vital for modern data-driven applications and business-critical services, where even marginal increases in latency can lead to substantial economic losses [16]. To improve query performance, a broad spectrum of configuration techniques have been extensively studied, including index selection [28, 36, 37, 43, 53], knob tuning [12, 26, 32, 44, 48, 49], query rewriting [21, 31, 55], and plan optimization via query hints [23, 38, 41, 57].

However, most existing approaches work separately, optimizing an individual component in isolation and are therefore insufficient to achieve globally optimal query performance. Although configurable components, such as knob tuning and indexing, span different layers of the query processing stack, they collectively determine the execution efficiency. As illustrated in Figure 1(a), tuning individual components yields limited gains, as improvements obtained from one component are often limited and could be offset by suboptimal settings in others. Consequently, multi-component tuning has attracted increasing attention [33, 45–47].

Despite its promise, multi-component tuning is fundamentally harder than single-component optimization for two key reasons:



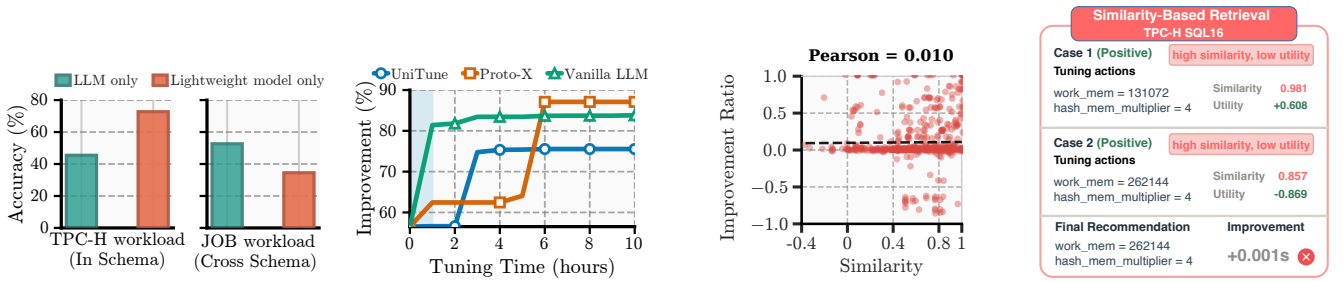
(a) Single vs. Multiple Tuning. **(b) Search Space Effect.**

Figure 1: Motivating Examples. (a) Multi-component tuning significantly outperforms tuning single component; (b) Existing methods fail to localize the anomaly subspace, leading to suboptimal tuning effectiveness under the same budget.

(1) combinatorially large search space, as indexes, knobs, hints, and query rewrites jointly create a massive configuration space that becomes intractable to explore exhaustively under limited budgets; ***(2) hard-to-learn tuning policies***, because the mapping from configurations to performance is much more complex and harder to learn efficiently than for single component. However, existing approaches [33, 45–47] do not adequately address either issue.

First, they rely on blind search rather than adaptive localization. Existing approaches are largely query-agnostic, as they do not leverage rich query information—such as SQL semantics, physical plans, and runtime statistics—to localize a query-specific anomaly subspace. Instead, they rely on generic search strategies: UDO [33] uses a hierarchical schedule that tunes indexes first and knobs later, UniTune [47] selects subspaces through trial-and-error based on sparse rewards, and Proto-X [45] and Booster [46] directly search the full joint configuration space. As a result, their search is either under-coordinated (e.g., UDO) or over-coordinated (e.g., Booster). In contrast, focusing on an anomaly subspace inferred from query information yields substantially higher efficiency under the same budget, as illustrated in Figure 1(b). ***Second, they remain inefficient in learning effective tuning policies.*** UniTune tunes configurations through ML-based online feedback optimization, which requires substantial exploration when rewards are sparse. UDO and Proto-X further depend on reinforcement-learning-based trial-and-error, requiring repeated interactions to optimize tuning decisions. As a result, these interaction-driven policy learners must accumulate many configuration-performance pairs, leading to substantial tuning overhead. These limitations expose two fundamental challenges for multi-component tuning: (i) how to effectively localize a promising subspace for search, and (ii) how to efficiently learn high-quality tuning policies under limited feedback.

How to search effectively? Instead of searching blindly without the guidance of query information and feedback, we propose to localize a query-specific anomaly subspace by jointly leveraging both. However, the query information is complex and implicit:



(a) Model Diagnosis Accuracy. **(b) LLM vs. Interaction-Driven.** **(c) Similarity Retrieval Scatter.** **(d) Similarity Retrieval Example.**

Figure 2: Preliminary Observations. (a) Lightweight models degrade under schema shifts, while LLMs remain stable but still unreliable across workloads; (b) LLM-guided tuning quickly identifies promising configurations but converges to suboptimal solutions; (c) Similarity is weakly correlated with improvement under similarity-based retrieval; (d) An example from TPC-H Q16 showing that highly similar retrieved cases may still provide limited utility for tuning.

existing anomaly diagnosis methods [25, 54] could learn execution patterns on one workload to infer anomaly-related components, but their accuracy often degrades noticeably under workload or schema shifts (detailed in Section 2.1). Moreover, tuning feedback is inherently partial: successes indicate promising space, but failures may stem from insufficient exploration rather than an incorrect subspace. *This raises the challenge: how to localize a promising tuning subspace from complex query information and partial feedback, and adapt it over time as new evidence becomes available.*

How to tune efficiently? Recent advances in large language models (LLMs) offer a few-shot paradigm for tuning [19, 34, 40], allowing promising actions to be inferred based on given context and reducing reliance on costly fine-tuning with large-scale interaction data. However, multi-component tuning is inherently iterative, so static few-shot inference [46] is insufficient (detailed in Section 2.1): tuning decisions must evolve as new execution feedback accumulates. This makes the quality and organization of observations critical. Low-quality observations can be uninformative or even harmful [9], while limited context windows prevent many valuable historical cases from being fully exploited. *This raises the challenge of how to select, organize, and continuously refine high-quality observations so that the LLM agent can support efficient tuning under context and feedback constraints.*

In this paper, we present EvoTune, a memory-aware evolution framework that localizes query-specific subspaces via collaborative diagnosis under partial feedback, and continuously refines tuning policies through utility-aware retrieval. To search effectively, EvoTune localizes the anomaly space from complex query information using two complementary diagnosticians: a lightweight data-driven model that captures recurring execution patterns, and an LLM-based Detection Agent that performs knowledge-guided reasoning for harder or out-of-distribution cases. A prior-guided bandit coordinates them, using the lightweight model as the default expert while invoking LLM reasoning and exploration when needed. To improve subsequent localization, EvoTune further refines the lightweight model with online tuning feedback under masked supervision, avoiding overfitting to unreliable partial signals. To tune efficiently, EvoTune treats historical-observation retrieval as a sequential decision problem rather than static similarity matching. It learns a utility-aware retriever from tuning trajectories to select observations that are truly useful under the current tuning state, while distilling recurring successful patterns into compact experiences to alleviate context-window limitations. Finally, EvoTune organizes tuning observations in a memory hub and uses execution feedback to jointly refine search and retrieval throughout the tuning process. We summarize our contributions as follows.

- We present EvoTune, an autonomous multi-component DBMS tuning framework that accelerates subspace localization and LLM-based policy learning with memory-aware evolution from execution feedback, without expensive LLM fine-tuning.
- We propose collaborative subspace localization via bandit-based expert selection to coordinate lightweight pattern-based prediction and LLM reasoning for reliable subspace localization.
- We propose a learned retrieval mechanism that selects utility-aware observations and distilled experiences to support effective LLM evolution for tuning action generation.
- Extensive experiments across workload and schema shifts show that EvoTune achieves up to 44.5% better performance than baselines under the same tuning budget, and reaches the final performance of the best competing baseline up to 3.9× faster.

2 Preliminary

We first present two motivating observations in Section 2.1, then formalize the multi-component tuning problem in Section 2.2.

2.1 Motivating Observations

In this subsection, we present two key observations that reveal fundamental gaps in effective search and efficient tuning, motivating a unified framework for robust localization and effective tuning observation utilization.

Observation 1: Neither Lightweight Models nor LLMs Diagnose Reliably. Existing methods [25, 42, 54] rely on lightweight ML models to diagnose performance issues by learning recurring patterns from historical observations. However, such models fundamentally assume stable data distributions. As shown in Figure 2(a), while they achieve high accuracy on in-distribution workloads (TPC-H), their performance degrades sharply under schema shifts (JOB), revealing limited generalization. LLMs, in contrast, perform

diagnosis more like human experts by reasoning over SQL semantics, execution plans, and runtime context, enabling better generalization to out-of-distribution cases. However, such reasoning is inherently non-statistical and fails to capture recurring execution patterns, leading to inconsistent and sometimes misleading diagnoses. These observations reveal a fundamental gap: lightweight models capture patterns but lack robustness, while LLMs generalize better but fail to exploit structured regularities. *This calls for a hybrid subspace localization strategy that combines pattern-based priors with selective LLM reasoning.*

Observation 2: Static-Similarity Retrieval Is Insufficient. We observe a complementary trade-off between LLM-based tuning and data-driven policy learning. LLMs quickly identify promising configurations but often plateau at suboptimal solutions, while existing interaction-driven methods improve more slowly yet eventually achieve better performance (Figure 2(b)). A natural approach is to combine these advantages by leveraging LLMs with feedback-driven refinement. However, fine-tuning LLMs on the feedback data is expensive and brittle under schema or workload shifts, making in-context learning the practical alternative. In this setting, tuning decisions are conditioned on retrieved historical cases, which directly influence the generated actions. As a result, tuning performance becomes highly sensitive to retrieval quality.

One approach is to retrieve similar past cases based on query similarity, typically measured by cosine distance [46]. However, similarity is not aligned with the utility of historical observations. As shown in Figures 2(c) and 2(d), similarity is nearly uncorrelated with the final improvement, and even highly similar cases often yield negligible or negative gains. This misalignment arises for two key reasons. First, query similarity is difficult to measure accurately for tuning, and commonly used metrics such as cosine similarity fail to capture true performance relevance. Second, tuning is inherently state-dependent: the usefulness of a historical case depends on the current tuning state, explored subspace, and past actions. Consequently, similarity-based few-shot inference remains static and fails to adapt to the evolving tuning context. *This reveals a fundamental limitation: effective tuning requires utility-aware retrieval that estimates the long-term benefit of historical cases under the current tuning state, rather than relying on static similarity.*

2.2 Problem Formulation

We aim to find an optimal configuration for a given query, particularly for slow SQL and OLAP workloads.

Tuning Component. For a given query, the tuning search space consists of four classes of actions: (1) *query rewriting*, which restructures SQL statements into semantically equivalent forms but with higher performance; (2) *query-level knob tuning*, which adjusts execution-related DBMS parameters that affect query behavior and can be set on a per-query basis (e.g., *work_mem*); (3) *optimizer hints*, which inject explicit directives to guide the query optimizer to generate better query plans (e.g., *enable_seqscan*); and (4) *index selection*, which recommends indexes to accelerate query processing.

Multi-Component Tuning. Suppose there are m tunable components, each component corresponds to a configuration subspace Θ_i $(1 \le i \le m)$, and a configuration setting is denoted by $\theta_i \in \Theta_i$. The full configuration space is defined as the Cartesian product $\Theta = \Theta_1 \times \Theta_2 \times \cdots \times \Theta_m$. Given a target query q and tuning objective f, the multi-component tuning problem aims to find a configuration:

$$\arg \min_{(\theta_1,\ldots,\theta_m)\in\Theta} f(\theta_1, \ldots, \theta_m; q), \tag{1}$$

where $f(\cdot; q)$ measures the execution performance of query q under a given configuration, typically in terms of query execution time.

Two-Stage Multi-Component Tuning. Given a query q, the goal is to efficiently optimize performance within a high-dimensional configuration space. Instead of directly searching the full space Θ, we formulate the problem as a two-stage decomposition:

(1) Tuning Space Localization. The goal is to determine a subset of bottleneck-related components $\mathcal{B}$, i.e., components whose configuration adjustments can lead to measurable performance improvements for the target query. Formally, we define:

$$\mathcal{B} = \{b_1, b_2, \ldots, b_n\} \subseteq \{1, 2, \ldots, m\}, \qquad 0 < n \le m, \tag{2}$$

which induces a query-specific subspace $\Theta_{\mathcal{B}} = \prod_{i\in\mathcal{B}} \Theta_i$. Note that the subspace $\Theta_{\mathcal{B}}$ is not fixed and can be iteratively refined as new evidence becomes available.

(2) Tuning over Subspace. Given the localized subspace $\Theta_{\mathcal{B}}$, the objective is to optimize the configuration within this reduced space:

$$\arg \min_{(\theta_{b_1},\ldots,\theta_{b_n})\in\Theta_{\mathcal{B}}} f(\theta_{b_1}, \ldots, \theta_{b_n}; q). \tag{3}$$

3 Overview

Figure 3 presents the end-to-end workflow of EvoTune. It consists of three tightly coupled modules: (1) tuning space localization, (2) tuning action recommendation, and (3) memory-aware evolution. These modules are instantiated as sequential stages in an online feedback-driven loop, where observations continuously improve subsequent localization and recommendation.

Workflow. In the **offline** phase, EvoTune pretrains a lightweight detection model and a learned retriever to warm-start localization and recommendation modules. In the **online** phase, for each target query, (1) it first collects *query information*, including the SQL statement, execution plan, and runtime metrics. (2) This information is then passed to both the localization and recommendation stages. (3) The localization stage identifies the target tuning components, determining where optimization should focus. (4) Conditioned on the localized subspace, the recommendation stage retrieves relevant observations and experiences to generate and apply tuning actions. (5) The resulting outcome is written to a memory hub as a new observation, based on which the memory-aware evolution module updates both the localization and recommendation strategies. Tuning terminates when the query performance reaches a target threshold or when no improvement is observed over several consecutive iterations. Next, we describe the modules in detail.

(1) **Tuning Space Localization.** Given the collected *query information*, EvoTune identifies bottleneck-related tuning components $\mathcal{B}$ through (1.1) a prior-guided expert selection mechanism that coordinates three localization choices: (1.2) component scoring with the lightweight model, (1.3) agent reasoning with the Detection Agent, and random exploration. This mechanism balances efficiency, reasoning flexibility, and exploration, allowing EvoTune to rely on lightweight prediction for common cases, invoke agent reasoning

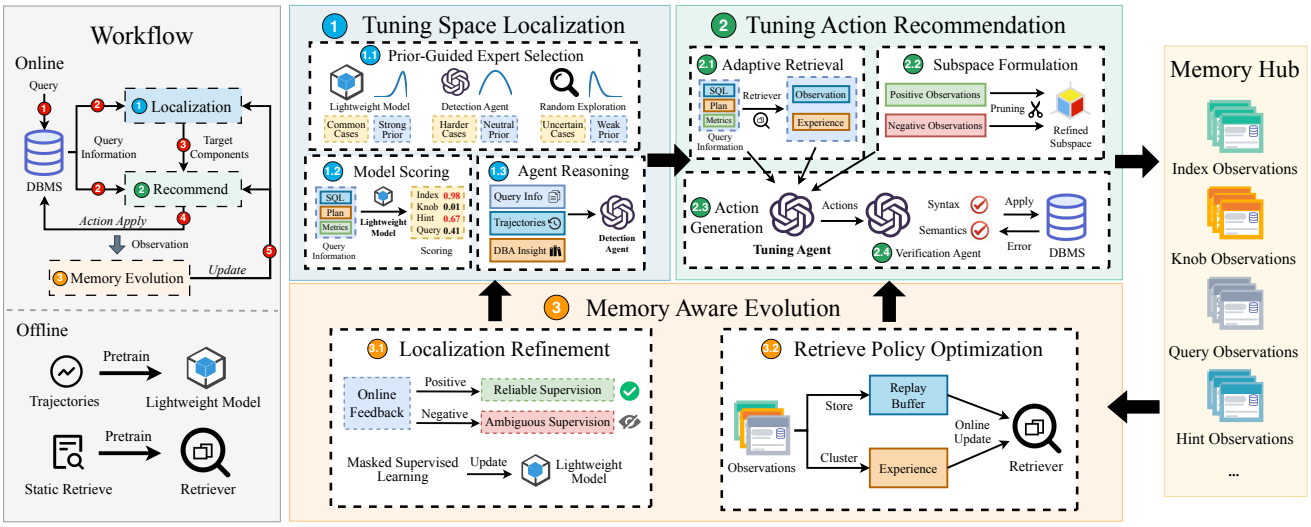


Figure 3: System Overview of EvoTune.

for cases that lightweight model fails to solve, and retain exploration when both become unreliable. The lightweight model takes the *query information* as input and estimates the likelihood that different tuning components exhibit performance bottlenecks; it is continuously refined by 3.1 using feedback from tuning outcomes. Guided by DBA insight and trajectories, i.e., previous tuning observations, the Detection Agent performs structured reasoning over the *query information* to infer the target tuning components.

2 **Tuning Action Recommendation.** Given the target tuning components, EvoTune recommends actions via a structured workflow. 2.1 *Adaptive Retrieval*: Retrieve historical observations with their experiences that are expected to be informative via a learned strategy (3.2) based on the target components and query information; 2.2 *Subspace Formulation*: Construct a refined subspace by pruning unpromising configurations to constrain search within a meaningful optimization region; 2.3 *Action Generation*: Generate tuning actions within the refined subspace by reflective reasoning over the retrieved history; 2.4 *Action Evaluation*: Validate the syntactic and semantic correctness of actions through interaction with a Verification Agent, and apply validated actions to evaluate query performance and update the memory hub with the observations.

3 **Memory-Aware Evolution.** To enable continuous adaptation, EvoTune introduces a memory-aware evolution mechanism that leverages tuning feedback to jointly optimize subspace localization and action recommendation modules through lightweight parameter updates, without fine-tuning the underlying LLMs.

3.1 *Feedback-Driven Localization Refinement.* To improve the subspace localization over time, EvoTune refines the lightweight model using feedback from tuning outcomes. Learning from online tuning feedback is challenging due to asymmetric and incomplete supervision: positive feedback provides reliable evidence for validated tuning components, whereas negative feedback is often ambiguous and may arise from insufficient exploration rather than incorrect localization. To address this issue, EvoTune applies masked supervised learning to ambiguous negative feedback, thereby avoiding incorrect hard supervision and enabling more reliable localization refinement over time (as detailed in Section 4.1.2).

3.2 *Retrieve Policy Optimization.* The effectiveness of tuning actions depends critically on the historical observations provided to the Tuning Agent, which reuses past feedback to guide recommendations. To make historical reuse adaptive under limited context windows, EvoTune clusters observations into distilled experiences (Section 5.1) and learns a utility-aware retriever (Section 5.2) to select informative observations and experiences for recommendation. Given the inherently multi-iteration nature of database tuning [44], EvoTune models the tuning process as a sequential decision problem and optimizes the retriever to maximize long-term utility under current state. As new observations are collected, they are stored in a replay buffer for retriever updates and clustered periodically into experiences for future recommendations.

4 Collaborative Detection

This section presents the collaborative localization mechanism, which combines a lightweight data-driven model (Section 4.1) and an LLM-based Detection Agent (Section 4.2) via an expert-based coordination (Section 4.3) for promising space localization.

4.1 Data-Driven Component Scoring

We first describe the construction of the lightweight detection model, followed by its localization refinement mechanism.

4.1.1 Component Scoring Model. We formulate tuning-component localization as a multi-label classification task over a predefined component set $C = \{1, \ldots, m\}$. Given the *query information* as input, the detection model predicts predicts component-level scores indicating the benefit of tuning each component. Formally, the detection model outputs component scores $\hat{\mathcal{Y}} = \{\hat{\mathcal{Y}}_1, \ldots, \hat{\mathcal{Y}}_m\}$, where

$\hat{\mathcal{Y}}_b \in [0,1]$ estimates the probability that the query performance can be improved by tuning the component b. The diagnosed component set $\mathcal{B}$ is obtained by thresholding $\hat{\mathcal{Y}}$ at 0.5, and could contain multiple components for compound anomalies.

Query Featurization. Given a target query, EvoTune collects its *query information* and represents it as a four-tuple $\mathcal{Q} = (S, P, I, E)$, capturing complementary aspects of query semantics, execution behavior, and system dynamics:

- *SQL Statement (S)* is treated as a textual sequence, where each token (e.g., `SELECT`) captures query intent as well as table or attribute references.
- *Execution Plan (P)* is represented as a structured execution trace composed of physical operators (e.g., sequential scan and join) together with their associated cost statistics and dependencies.
- *Internal Metrics (I)* record DBMS statistics collected during query execution. These metrics reflect query behaviors such as row access, block I/O, and buffer miss.
- *External Metrics (E)* are multivariate time series tracking system-level indicators (e.g., CPU utilization, memory load, I/O throughput). As query execution durations vary, we apply linear interpolation to align all time series to a uniform length.

We encode each modality using a dedicated encoder [25]: BERT [10] for query statements, QueryFormer [51] for execution plans, a multi-layer perceptron (MLP) for internal metrics, and a 2D-CNN model for external metrics. The resulting representations are fused via a cross-modal Transformer, followed by an MLP that outputs component-level logits.

Impact Label Definition. To supervise component scoring, we derive an outcome label indicating whether the selected components are worth tuning. Given a diagnosed component set $\mathcal{B}$, we define a binary tuning outcome after applying action $\mathcal{A}$ over $\mathcal{B}$:

$$\mathcal{Y} = \mathbb{I}\left[\frac{\mathcal{T}(\mathcal{Q}) - \mathcal{T}(\mathcal{Q}, \mathcal{A})}{\mathcal{T}(\mathcal{Q})} \geq \delta\right]. \tag{4}$$

Based on this outcome, the component-level impact label for each component $b \in \mathcal{C}$ is defined as:

$$\mathcal{Y}_b = \mathbb{I}[b \in \mathcal{B}] \cdot \mathcal{Y}. \tag{5}$$

Here, $\mathcal{T}(\mathcal{Q})$ and $\mathcal{T}(\mathcal{Q}, \mathcal{A})$ denote the query latency before and after applying action $\mathcal{A}$, respectively. δ is a minimum improvement threshold (set to 5%) to filter noise, and $\mathbb{I}[\cdot]$ denotes indicator function. Thus, components in $\mathcal{B}$ are labeled positive only if the action yields sufficient improvement.

4.1.2 Continual Refinement via Masked Supervision. A key advantage of the detection model is its ability to capture fine-grained patterns from data via lightweight parameter updates, compared with LLM agents. However, online tuning feedback is inherently asymmetric and incomplete for subspace localization. A positive outcome provides reliable evidence that tuning component b is beneficial for query i. In contrast, a negative outcome is often ambiguous: it may indicate that b is truly ineffective, but it may also simply result from insufficient exploration over that component. Thus, directly applying a standard binary cross-entropy (BCE) loss to all negative labels is inappropriate, because it would treat under-explored components as true negatives and discourages further exploration, introducing false-negative bias and suppressing potentially beneficial components.

To address this issue, we adopt a masked binary cross-entropy objective that retains only reliable supervision. Let $n_{i,b}$ denote the number of tuning trials on component b for query i, and let η be a minimum exploration threshold. A negative label is regarded reliable only when $n_{i,b} \geq \eta$. Besides, components not selected for tuning are treated as unlabeled. Accordingly, we define the supervision mask as:

$$m_{i,b} = \begin{cases} 1, & \mathcal{Y}_{i,b} = 1, \\ 1, & \mathcal{Y}_{i,b} = 0 \ \wedge \ n_{i,b} \geq \eta, \\ 0, & \mathcal{Y}_{i,b} = 0 \ \wedge \ n_{i,b} < \eta, \\ 0, & b \notin \mathcal{B}. \end{cases} \tag{6}$$

We minimize the following masked BCE loss:

$$\mathcal{L}_b = -\frac{1}{\sum_i \sum_{b=1}^{m} m_{i,b}} \sum_i \sum_{b=1}^{m} m_{i,b} \left[\mathcal{Y}_{i,b} \log \hat{\mathcal{Y}}_{i,b} + (1 - \mathcal{Y}_{i,b}) \log(1 - \hat{\mathcal{Y}}_{i,b})\right] \tag{7}$$

In this way, EvoTune always learns from verified positive signals and sufficiently explored negative signals, while ignoring unreliable negatives caused by limited exploration.

4.2 Knowledge-Driven Reasoning

The Detection Agent infers tuning components with domain knowledge and historical trajectories to ensure robust diagnosis.

Domain Knowledge-Guided Reasoning. We encode DBA insights as structured prompts to emulate DBA-style reasoning for bottleneck validation. Representative insights include:

- *Plan Suboptimality.* A plan bottleneck typically arises from suboptimal access paths, join orders, or join strategies that introduce unnecessary overhead, such as redundant sorting, full table scans, or excessive intermediate results. Typical evidence includes cases where the optimizer prefers sequential scans despite the presence of indexes under high selectivity, incurs extra sorting due to unordered access paths, exhibits hash join spill-to-disk behavior.
- *Index Bottleneck.* Index bottlenecks are characterized by large-range scans on selective predicates. Typical evidence includes a large gap between scanned and returned rows, predicates that cannot be effectively exploited by indexes, and repeated heap/table access due to non-covering indexes.
- *Knob Misconfiguration.* Knob tuning becomes important when analytical operators spill to disk or lack sufficient memory or parallel resources. Typical evidence includes disk-backed temporary tables, underutilized parallel workers, and high-cost join or aggregation execution under limited resources.
- *Query Formulation Issues.* Query rewriting is prioritized when the SQL structure itself introduces unnecessary work. Typical evidence includes implicit type conversions (blocking B-Tree access and triggering full scans), overlong `IN` lists.

Trajectory-Guided Reasoning. For each query i, we leverage its tuning trajectory to refine component confidence and avoid repeatedly exploring unpromising directions. Specifically, for each component b, EvoTune maintains the numbers of successful and failed tuning attempts, denoted by $s_{i,b}$ and $f_{i,b}$. These statistics are incorporated into structured prompts to guide the Detection Agent according to the following principles:

- If $s_{i,b} > 0$ and $f_{i,b} = 0$, prioritize component b, since past interventions consistently validate it as an actionable bottleneck.
- If $s_{i,b} = 0$ and $f_{i,b} > 0$, deprioritize component b and encourage exploration of alternative components, since previous interventions on b have not led to improvement.
- If $s_{i,b} > 0$ and $f_{i,b} > 0$, keep component b under consideration and let the Detection Agent make a more nuanced judgment based on DBA knowledge and the available evidence.

Based on the above knowledge and trajectory signals, we construct a structured prompt and populate it with database context and selected query information (listed in Appendix A.1 [4]), based on which the Detection Agent infers the tuning components $\mathcal{B}$.

4.3 Prior-Guided Expert Selection

As discussed in Section 2.1, the two diagnosis paradigms exhibit complementary strengths for tuning-space localization: the *lightweight detection model* efficiently captures recurring patterns from historical data, while the *LLM-based Detection Agent* reasons more flexibly over query evidence and is thus better suited for harder or out-of-distribution cases when lightweight model fails.

To address this uncertainty, we adaptively select the most suitable diagnostician for each query by formulating diagnostician selection as a multi-armed bandit problem [7], where each expert is treated as an arm and selected at each tuning step. To encode prior knowledge about the relative strengths of different experts, we incorporate asymmetric priors, guiding selection using both prior knowledge and downstream tuning feedback.

Expert Arms. We define three expert arms $E = \{e_{\text{lm}}, e_{\text{llm}}, e_{\text{rand}}\}$, where e_{lm}, e_{llm}, and e_{rand} denote the lightweight model, the Detection Agent, and random exploration, respectively. The random arm serves as a fallback to explore alternative subspaces when both diagnosticians are uncertain, preventing overcommitment to potentially suboptimal regions.

Prior-Guided Expert Selection. For each arm $e \in E$, EvoTune maintains a Beta distribution $\theta_e \sim \text{Beta}(\alpha_e, \beta_e)$ to model its expected utility for tuning-space localization, where α_e and β_e correspond to the accumulated counts of positive and negative feedback. EvoTune adopts Thompson sampling for expert selection: at each tuning iteration t, it draws $\tilde{\theta} \sim \text{Beta}(\alpha_e, \beta_e)$ for each arm and selects $e_t = \arg\max_{e \in E} \tilde{\theta}_e$. To encode prior knowledge about the relative strengths of different diagnosticians, the initialization is deliberately asymmetric: the lightweight model is assigned the strongest prior as it achieves higher accuracy on in-distribution workloads, the Detection Agent receives a weaker but still competitive prior for harder or out-of-distribution cases, and random exploration receives the weakest prior. Accordingly, the initialization satisfies $\mathbb{E}[\theta_{e_{\text{lm}}}] > \mathbb{E}[\theta_{e_{\text{llm}}}] > \mathbb{E}[\theta_{e_{\text{rand}}}]$, which is achieved by using Beta distributions with different parameterizations. In practice, we use $\text{Beta}(8, 2)$, $\text{Beta}(2, 2)$, and $\text{Beta}(2, 8)$ for the three arms, respectively.

Feedback-Driven Bandit Update. After selecting expert e_t, EvoTune optimizes within the localized subspace $\mathcal{B}_t$ for η iterations, rather than immediately performing another expert selection, as a promising subspace may require multiple iterations to reveal its utility and would otherwise be prematurely discarded. Then, EvoTune summarizes the downstream outcome of $\mathcal{B}_t$ into a binary impact label $\mathcal{Y}_t$ (Eq. 4). The posterior parameters of the selected arm are then updated as:

$$\alpha_{e_t} \leftarrow \alpha_{e_t} + \mathcal{Y}_t, \qquad \beta_{e_t} \leftarrow \beta_{e_t} + (1 - \mathcal{Y}_t). \tag{8}$$

Thus, experts that repeatedly yield effective localization decisions receive stronger posterior support, while those leading to weak localization outcomes are gradually down-weighted. To ensure query-specific adaptation, the Beta distributions are re-initialized for each query.

5 Tuning Action Recommendation

The tuning process can be naturally formulated as a Markov Decision Process (MDP), since tuning is inherently a sequential decision-making problem in which each action affects both the immediate reward and the subsequent tuning state [44]. When leveraging LLM agents for tuning, particularly in retrieval-augmented settings, we argue that tuning decisions are conditioned not only on the current state but also on retrieved historical cases, which encode past tuning experiences and provide essential context for decision making. Formally, at iteration t, we define the current state as $\mathcal{S}_t = (\mathcal{Q}_t, \mathcal{B}_t)$, where $\mathcal{Q}_t$ denotes the current query information and $\mathcal{B}_t$ the localized tuning components. The memory hub $\mathcal{M}_t = \mathcal{O}_1, \ldots, \mathcal{O}_N$ stores historical tuning observations accumulated up to iteration t. For LLM agents, a retrieval policy $\mu(\mathcal{O} \mid \mathcal{S}_t, M_t)$ selects relevant past observations, based on which the LLM generates actions following $p_{\text{LLM}}(\mathcal{A}_t \mid \mathcal{S}_t, \mathcal{O})$. The overall tuning policy is thus given by:

$$\pi(\mathcal{A} \mid \mathcal{S}_t, M_t) = \sum_{\mathcal{O} \in M_t} \mu(\mathcal{O} \mid \mathcal{S}_t, M_t)\, p_{\text{LLM}}(\mathcal{A} \mid \mathcal{S}_t, \mathcal{O}). \tag{9}$$

To improve the tuning policy, p_{LLM} is instantiated with an off-the-shelf LLM, enabling seamless upgrades as more advanced models become available. Rather than relying on LLM fine-tuning, which is costly and prone to overfitting under distribution shifts [39], we focus on improving decision quality through external memory and retrieval. This highlights the central role of memory construction and retrieval in tuning. Accordingly, EvoTune first organizes historical feedback into a hierarchical memory hub (Section 5.1), then learns an adaptive retrieval policy over this memory (Section 5.2), and finally generates and validates executable actions (Section 5.3).

5.1 Experience Distillation and Evolution

To support the sequential tuning process with reusable memory, EvoTune organizes historical feedback into a hierarchical memory hub with two levels: *instance-level observations* and *cluster-level experiences*. The former preserve fine-grained evidence from individual tuning attempts, while the latter capture stable action patterns shared across similar observations. This two-level design enables effective reuse by combining query-relevant evidence with higher-level, transferable tuning guidance.

Observation Construction. After each tuning iteration, EvoTune records the tuning result as an instance-level observation

$$\mathcal{O} = (\mathcal{Q}, \mathcal{B}, \mathcal{A}, \mathcal{T}, \mathcal{Y}, \mathcal{F}), \tag{10}$$

where $\mathcal{Q}$, $\mathcal{B}$, $\mathcal{A}$, and $\mathcal{T}$ denote the query information, localized tuning components, applied action, and observed execution performance, respectively, and $\mathcal{Y} \in 0, 1$ denotes the outcome label defined in Eq. 4. To improve transferability while keeping historical feedback grounded, EvoTune further derives a reflection $\mathcal{F}$ from the

current tuning result by abstracting the query and execution characteristics, the applied action, and the resulting outcome. According to label $\mathcal{Y}$, the reflection $\mathcal{F}$ is categorized into two forms:

- *Positive reflection*: captures why the applied action is effective under the observed query and execution evidence.
- *Negative reflection*: abstracts why the applied action is ineffective under the observed query and execution evidence.

Experience Distillation. While reflections capture fine-grained feedback for individual observations, they remain tied to specific cases. EvoTune therefore further abstracts observations into experiences for more transferable reuse. To avoid noisy or overly case-specific summaries, it first clusters similar observations and then distills each cluster into an experience that captures a stable tuning pattern, including representative execution characteristics, recurring actions, and their overall effects. For example, EvoTune distills the following experiences from historical observations:

- *Positive experience*: when execution remains scan-dominated and the number of planned/launched workers has already reached the limit of `max_parallel_workers_per_gather`—the PostgreSQL parameter that caps the maximum number of parallel workers used by a single gather node—increasing this limit may help by enabling more parallel scan workers.
- *Negative experience*: when the dominant cost comes from repeated large-scale aggregations and rescans, and the relevant sort/hash operators already fit in memory without spilling to disk, simply increasing `work_mem`—the per-operator memory budget for in-memory sorting and hashing—may fail to improve performance, as the bottleneck is not memory-bound but stems from the data-intensive execution pattern.

To support clustering, EvoTune represents each observation using a joint embedding constructed by concatenating the query information embedding (Section 4.1.1), the action embedding produced by a BERT encoder, and a scalar latency-improvement feature. Considering that online tuning observations are inherently heterogeneous and noisy, and the number of underlying latent tuning patterns is unknown in advance, EvoTune adopts HDBSCAN [24] for observation clustering. HDBSCAN can identify clusters of varying densities without requiring a predefined number of clusters, while leaving noisy observations unassigned rather than forcing them into unreliable clusters. For each cluster, EvoTune invokes a Summary Agent that summarizes the reflections derived from its member observations into a single cluster-level experience.

Experience Evolution. As tuning observations arrive continuously during online tuning, EvoTune incrementally maintains the experience instead of reclustering all observations from scratch. For each cluster, EvoTune keeps a prototype, defined as the average embedding of its member observations, and a radius threshold, defined as the distance from the prototype to the farthest member observation. Given a new observation, EvoTune computes its distance to all cluster prototypes and assigns it to the nearest cluster if the distance falls within that cluster's radius. In that case, the observation is merged into the cluster. Otherwise, it is stored in a *pending buffer*. When the buffer accumulates sufficient observations, EvoTune reclusters them to identify emerging tuning patterns that are not captured by existing clusters.

5.2 Adaptive Memory Retrieval

Given the memory hub, the central challenge is to retrieve historical observations that are most beneficial for the tuning process. Due to the sequential nature of tuning as an MDP, the utility of a retrieved observation cannot be determined in isolation, but depends on its long-term contribution to tuning performance. The observations to be retrieved should evolve as the tuning proceeds to support continual adaptation of the agent's tuning policy $p_{\text{LLM}}(\mathcal{A}_t \mid \mathcal{S}_t, O)$, rendering static retrieval inadequate. Therefore, we propose a reinforcement learning formulation to optimize retrieval decisions based on their long-term impact on tuning performance.

To this end, EvoTune leverages tuning feedback to train a parameterized retriever that learns a Q-function over current state and historical observations, which guides retrieval by estimating their expected utility for long-term gain. Importantly, the current state is not limited to the SQL statement, but also includes execution plans and runtime metrics, which are continuously shaped by previously applied actions and observed feedback. The retriever is optimized via soft Q-learning to prioritize observations that maximize long-term utility for iterative tuning. We define the reward as the immediate reduction in query latency after applying the action induced by the retrieved observation:

$$\mathcal{R}_t = \mathcal{T}(\mathcal{S}_t) - \mathcal{T}(\mathcal{S}_{t+1}),\ \mathcal{A}_t \sim p_{\text{LLM}}(\cdot \mid \mathcal{S}_t, O_t),\ \mathcal{S}_{t+1} = f(\mathcal{S}_t, \mathcal{A}_t), \quad (11)$$

where O_t denotes the retrieved observation at iteration t. The Q-value captures the expected cumulative performance improvement when selecting an observation under the current state. Specifically, for each candidate historical observation, the learned retriever takes as input a feature vector formed by concatenating: (i) the query information embedding of Q_t, (ii) a binary encoding of the diagnosed components $\mathcal{B}_t$, and (iii) the embedding of the candidate observation. The retriever is implemented as a two-layer MLP that outputs a scalar Q-value.

Inspired by [52], we adopt a maximum-entropy reinforcement learning objective [13] to optimize the retrieval policy while encouraging diversity among retrieved observations.

$$J(\mu) = \mathbb{E}_{p_\mu}\left[\sum_{t=0}^{T-1} (\mathcal{R}_t + \mathcal{H}(\mu(\cdot \mid \mathcal{S}_t, M_t)))\right], \quad (12)$$

where $\mathcal{H}$ denotes the entropy. Under the maximum-entropy objective in Eq. 12, the optimal retrieval policy induced by the soft Q-function takes a Boltzmann form over candidate observations:

$$\mu^*(O \mid \mathcal{S}_t, M_t) = \frac{\exp\left(Q^*(\mathcal{S}_t, M_t, O)\right)}{\sum_{O' \in M} \exp\left(Q^*(\mathcal{S}_t, M_t, O')\right)}, \quad (13)$$

it assigns larger retrieval probabilities to observations with higher Q-values. Using tuning trajectories, the retriever is trained by minimizing temporal-difference (TD) loss. Specifically, given a transition $(\mathcal{S}_t, M_t, O_t, \mathcal{R}_t, \mathcal{S}_{t+1}, M_{t+1})$, the state value under the optimal retrieval policy is defined as:

$$V^*(\mathcal{S}_t, M_t) = \log \sum_{O' \in M_t} \exp\left(Q^*(\mathcal{S}, M_t, O')\right), \quad (14)$$

which measures the maximum long-term utility achievable at retrieval state $(\mathcal{S}_t, M_t)$ when candidate observations are selected according to the optimal retrieval policy. The TD target is defined using a soft Bellman backup: $Q_t^{\text{target}} = \mathcal{R}_t + \gamma V(\mathcal{S}_{t+1}, M_{t+1})$. Here,

$\gamma \in [0, 1)$ is the discount factor for future retrieval utility. The Q-network is optimized by minimizing the squared TD error:

$$\mathcal{L}_{\mathrm{TD}}(\theta) = \mathbb{E}\Big[\big(Q_\theta(\mathcal{S}_t, M_t, O_t) - Q_t^{\mathrm{target}}\big)^2\Big]. \tag{15}$$

Offline Pretraining. Learning a promising retrieval policy from a randomly initialized network is challenging, as in the initial training stage the retriever tends to select uninformative observations, leading to sparse reward signals and hindering stable and efficient convergence. To mitigate this issue, we adopt an offline training strategy by first constructing trajectories using a similarity-guided bootstrap policy. Specifically, with probability $1 - \epsilon$, the bootstrap policy selects an observation from the top-k most similar candidates, and with probability ϵ, it samples an observation uniformly at random from the remaining memory. Although similarity is insufficient as the final retrieval criterion, it serves as a useful weak prior for guiding early exploration, while random exploration improves coverage over historical observations and prevents the retriever from collapsing to a purely similarity-based policy.

Online Continual Learning. During online retrieval, the retriever selects the top-k observations with the highest Q-values, together with the associated cluster-level experiences to construct a compact yet informative context for the Tuning Agent. Newly collected tuning trajectories are continuously stored in a replay buffer. Once the buffer reaches 256 samples, the Q-network is updated using randomly sampled batches to further minimize the TD loss in Eq. 15, enabling continual refinement of the retrieval policy under evolving query contexts.

5.3 Action Generation and Evaluation

Given the retrieved observations $\mathbb{O} = \{O_1, \ldots, O_k\}$, EvoTune follows a structured workflow to produce tuning actions: it constructs a refined tuning subspace to guide recommendation in promising regions, and then generates candidate actions with reflective reasoning and validates them through execution-oriented evaluation.

Subspace Formulation. Before action generation, EvoTune first defines a *basic tuning space* from the localized tuning components $\mathcal{B}$: $\Theta_{\mathcal{B}} = \prod_{b \in \mathcal{B}} \Theta_b$, which provides the Tuning Agent with the allowable operation space for each identified component. Based on this basic space, EvoTune further leverages the retrieved observations to refine it into a *promising subspace* Θ^* by pruning ineffective choices and narrowing the search toward historically beneficial regions. Specifically, for discrete spaces such as indexes and query rewrites, EvoTune prunes candidates with repeatedly ineffective outcomes in retrieved observations; for continuous or configurable spaces such as knobs and hints, it narrows the admissible values to regions that previously achieved higher utility. Importantly, Θ^* serves as a guidance prior rather than a hard constraint, allowing the Tuning Agent to deviate from it when necessary. Details are deferred to Appendix A.2 [4].

Generation with Reflective Reasoning. Conditioned on $\mathcal{Q}$, $\mathcal{B}$, $\mathbb{O}$, and Θ^*, the Tuning Agent generates tuning actions via *reflective reasoning* [27]. Specifically, at each iteration t, EvoTune prompts the agent to reflect on the outcomes of previous attempts, analyze why past actions in retrieved observations succeeded or failed, and produce the next evidence-supported action: $\mathcal{A}_{t+1} \sim p_{\mathrm{LLM}}(\mathcal{Q}_t, \mathcal{B}_t, \mathbb{O}_t, \Theta_t^*)$. To ensure executability, EvoTune requires the generated action to follow predefined templates for each tuning component, together with DBMS-specific syntax constraints, so that it can be directly applied to the target system.

Action Evaluation Pipeline. EvoTune uses the Verification Agent to validate each generated action for syntactic validity and execution feasibility. Specifically, the agent checks whether the action conforms to the DBMS-specific syntax and whether it can be successfully applied or executed on the target system. Invalid actions are retried within a fixed attempt budget, and persistent failures are recorded in the memory hub to avoid repeated invalid attempts in future iterations. For query rewriting, EvoTune further verifies semantic equivalence with the original SQL through a hierarchical, tool-first pipeline. It first applies QED [35], a semantic equivalence checker for rewritten SQL. If QED is inconclusive, EvoTune executes both queries and compares their results. If a mismatch remains after up to n rounds of iterative repair, EvoTune falls back to the original SQL. After execution, EvoTune assigns the outcome label $\mathcal{Y}$ and generates the reflection $\mathcal{F}$, then stores the resulting observation $(\mathcal{Q}, \mathcal{B}, \mathcal{A}, \mathcal{T}, \mathcal{Y}, \mathcal{F})$ in the memory hub.

6 Evaluation

We detail the experimental setup in Section 6.1 and present the end-to-end evaluation in Section 6.2. We then validate the design of EvoTune, including space localization in Section 6.3, action recommendation in Section 6.4, followed by evaluations under additional scenarios in Section 6.5. Due to space constraints, we defer prompt templates and additional results, including API selection, overhead analysis, and ablation of subspace formulation to appendix [4].

6.1 Experimental Setup

Deployment Setting. To evaluate the generality of EvoTune across different database engines, we conduct experiments on two open-source database systems, PostgreSQL and TiDB. We evaluate EvoTune on PostgreSQL deployed on a cloud ECS machine equipped with 16 vCPUs, 64GB of RAM, and one NVIDIA RTX 5880 Ada Generation GPU featuring 48GB of memory. The *pg_hint_plan* [3] extension is used to enable hint-based plan tuning, and QED [35] is adopted for query equivalence checking. The experimental setup for TiDB is described separately in Section 6.5.2. Both databases are initialized using the default knob settings recommended in the official deployment manual, with only primary key indexes created and no query hints applied.

Workloads. We evaluate three analytic benchmarks: (1) JOB [18], 113 queries based on real workloads over the IMDB dataset; (2) TPC-H SF10 [2], a decision-support benchmark containing 22 analytical queries. (3) TPC-DS SF50 [1], a data-warehousing benchmark with 99 complex queries over 25 tables. These benchmarks cover diverse data scales and workload patterns.

Tuning Space. We construct the knob space using 27 query-level tunable knobs (e.g., max_parallel_workers, work_mem) and use 19 query hints (e.g., enable_hashagg, enable_sort) to form hint space. For index space, EvoTune considers indexes over all database columns including composite indexes. For query rewrite, we consider 70 rewrite rules augmented with descriptions as guidance [30].

Baselines. We compare two categories of baselines: *single-component methods* and *multi-component methods*, covering both specialized

tuners and unified tuning frameworks. For fair comparison, all LLM-based baselines use GPT-4.1 as the backend model.

Single-Component Baselines. We include four representative tuning methods, each specialized for a single tuning component:

- **LLMIA** [50]: an LLM-based index advisor that recommends index configurations based on workload and database features.
- **GPTuner** [17]: an LLM-enhanced knob tuning system that leverages structured knowledge extracted from DBMS manuals and related text to guide workload-aware knob selection with coarse-to-fine Bayesian tuning optimization.
- **R-Bot** [31]: an LLM-based rewriting system that retrieves rewrite evidence and iteratively refines rewriting decisions.
- **AutoSteer** [5]: a learned query hint recommendation system that automatically discovers effective hint sets to steer the optimizer.

Multi-Component Baselines. We compare EvoTune against existing multi-component tuning baselines:

- **Sequential Tuning**: A composite baseline that combines LLMIA, GPTuner, AutoSteer, and R-Bot to optimize all four components respectively in a sequential order. For fairness, the total tuning budget is evenly divided across the four components.
- **UDO** [33]: A system that relies on *predefined tuning schedules*. It separates heavy parameters (e.g., indexes) and light parameters (e.g., knobs), and optimizes them with different reinforcement learning strategies. It formulates tuning as a two-level process, where heavy parameters are optimized first, followed by the optimization of light parameters conditioned on the selected heavy-parameter configuration.
- **UniTune** [47]: A framework that coordinates multiple tuners for different components and selects which tuner to execute based on observed feedback.
- **Proto-X** [45]: A system that treats all configurable components as a unified search space and employs an actor-critic model to synthesize tuning actions in a single step. It captures structural and objective similarities across heterogeneous configuration spaces in a latent space to improve search efficiency.
- **Booster** [46]: An LLM-based framework that adopts a retrieval-augmented generation (RAG) paradigm, organizing past observations into query-level contexts and retrieving relevant historical cases for each target query based on similarity.

Evaluation Protocol. We set the overall tuning budget to 30 hours and impose a 5-minute timeout for each query execution. Queries with execution latency below 1 second are considered normal and excluded from the tuning process. We report the average query latencies as the evaluation metric, with each result averaged over 3 trials. For fair comparison, we follow Proto-X to execute the workload queries serially during evaluation.

EvoTune Setting. For the tuning space localization module, we adopt the model architecture of RCRank [25] for the lightweight model and continuously update it with a masked BCE loss on incrementally collected data, using a batch size of 8 (Section 4.1.2). For the tuning action recommendation module, we set $k = 4$ for top-k observation selection, since prior studies [6] show that selecting 3–5 observations provides a practical balance between diversity and relevance. To preserve contrastive richness, we limit the selection to at most 2 positive and 2 negative observations. For the Verification Agent, we implement a maximum retry budget of 3 times. For the learned retriever, we set the discount factor γ to 0.99 and update the Q-network using randomly sampled mini-batches of 64 samples. To organize the experience, we cluster observations using HDBSCAN ($min_cluster_size = 5$, $min_samples = 5$), and maintain a pending buffer of size 20 to trigger periodic re-clustering.

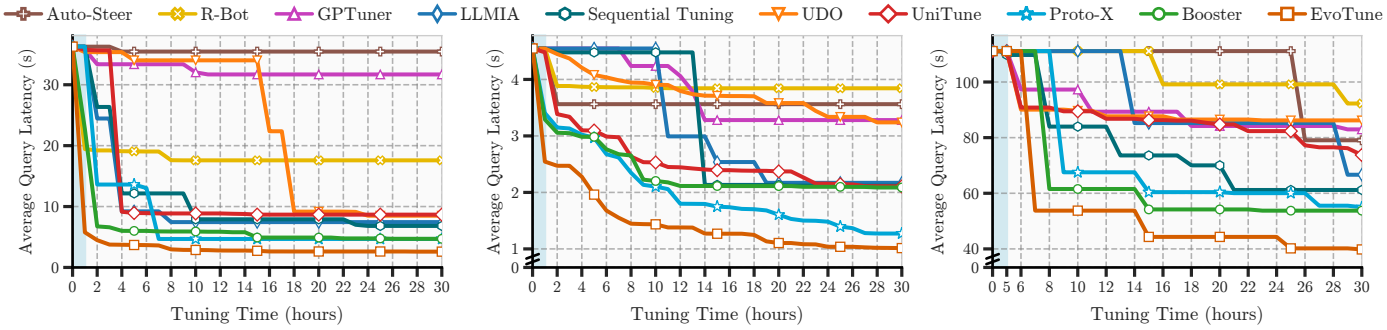
Offline Preparation. EvoTune collects tuning trajectories (30 hours, 3,385 samples), which are used to pre-train the component scoring model (Section 4.1.1) and the retrieve model (Section 5.2) on synthetic workloads derived from TPC-H schemas. To evaluate whether the two models in EvoTune can generalize to other scenarios, we consider both workload shift and schema shift settings in Section 6.2. Specifically, we evaluate on the standard TPC-H workload (workload shift), as well as JOB and TPC-DS workloads (schema shift). We further report a cold-start setting (Section 6.5.2), where neither model is pre-trained and both are updated purely online during tuning, to evaluate their continuous learning ability under more challenging scenarios. For Proto-X, we follow its original paper [45] and construct the latent space using 8192 samples. In contrast, the memory hub in both EvoTune and Booster is initialized empty to ensure a fair comparison and is progressively populated during tuning.

6.2 End-to-end Comparison

We first report the end-to-end comparison result. As illustrated in Figure 4, EvoTune consistently outperforms all baseline approaches across all workloads under both workload shift and schema shift settings. Single-component baselines are effective only when the dominant bottleneck lies within their optimization scope, leading to limited performance gains. This highlights that, in realistic workloads, performance bottlenecks often span multiple components, underscoring the necessity of multi-component tuning.

Analysis of Multi-Component Baselines. Sequential Tuning evenly allocates a fixed tuning budget across components, wasting efforts on ineffective subspaces and leading to suboptimal exploration when bottlenecks vary across queries. Among multi-component baselines, UDO performs the worst due to its fixed tuning schedule, which spends a large portion of the budget exploring the knob space regardless of query characteristics, making it unsuitable for diverse workloads. UniTune shows limited performance in the early stage, especially in TPC-DS, as it relies on random subspace selection before sufficient feedback is accumulated. Proto-X suffers from the curse of dimensionality, requiring extensive trial-and-error exploration in a large tuning space.

Despite being an LLM-based baseline, Booster performs significantly worse than Proto-X on the JOB workload. This is because it relies on high-quality historical samples to construct its memory (in its original design [46], such samples are generated by running an ML-based tuner for hours), but lacks the ability to effectively explore the tuning space and continuously evolve its policies. As a result, it tends to prematurely converge to configurations that are only marginally better than those stored in its memory, rather than discovering substantially better configurations. In our setting, where all methods start from an empty memory for fairness, this tendency toward premature convergence becomes more pronounced. In contrast, EvoTune incorporates evolution into both subspace exploration and action recommendation by iteratively



(a) TPC-H (Workload Shift) **(b) JOB (Schema Shift)** **(c) TPC-DS (Schema Shift)**

Figure 4: Best-so-far average query latency (lower is better) versus tuning time.

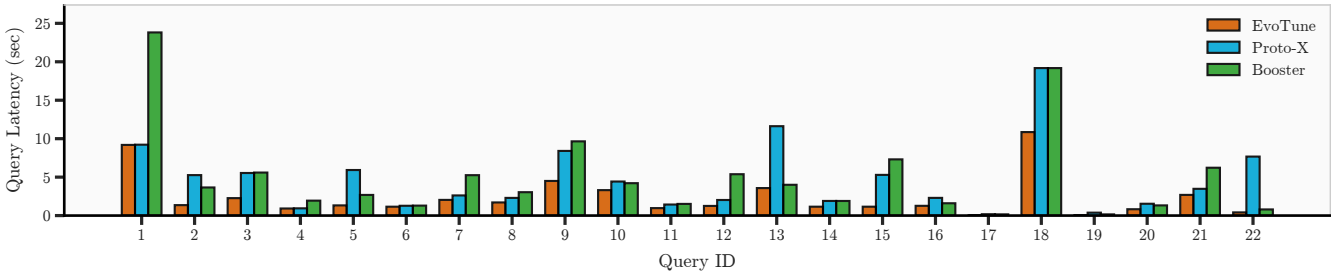


Figure 5: Per-query performance comparison on the TPC-H workload.

refining the search space and dynamically retrieving high-utility experiences, enabling more effective exploration and progressively improving decisions. Under the same 30-hour tuning budget, EvoTune reduces the final average query latency by 20.3%, 44.5%, and 27.7% on JOB, TPC-H, and TPC-DS, respectively. Compared with the strongest baseline on each workload—Proto-X on JOB and TPC-H, and Booster on TPC-DS—EvoTune reaches the same final performance 2.1×, 3.9×, and 1.5× faster, respectively.

Per-Query Performance Analysis. We further analyze per-query execution time on the TPC-H benchmark. EvoTune is particularly effective at reducing the execution time of long-tail queries (e.g., Q5, Q9, and Q15). In contrast, Proto-X and Booster exhibit persistently high execution times on these complex queries, indicating their inability to resolve specific bottlenecks. By better coordinating multiple tuning components, EvoTune substantially improves the performance of these hard queries.

Overhead Analysis. The tuning overhead can be divided into two parts: (1) *algorithm overhead* and (2) *replay overhead*. For algorithm overhead, EvoTune spends 3.8 s on diagnosis, 3.7 s on tuning action recommendation, 1.0 s for lightweight model refinement and 0.6 s on online retriever update on average for each SQL. Although EvoTune may incur higher algorithm overhead than ML-based tuners, it reaches the same target performance with substantially lower replay cost. To measure the efficiency, we define a target as achieving 80% of EvoTune's best performance, as higher targets are often unattainable for other methods, and measure the time required to reach this target among baselines. On TPC-H, EvoTune incurs 2.4× higher algorithm time than Proto-X to reach the target,

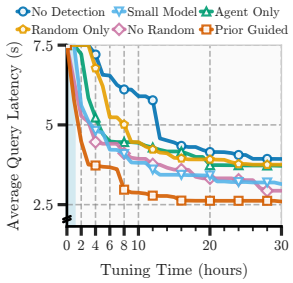


Figure 6: Detection Analysis.

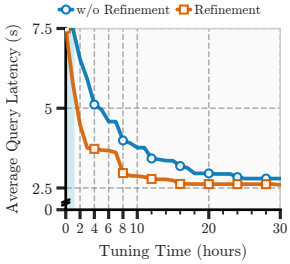


Figure 7: Refine Analysis.

but reduces replay time by 2.5×, resulting in a 2.1× shorter total time-to-target. In comparison to Booster, EvoTune uses 3.8× less algorithm time and 1.9× less replay time, yielding a 2.3× shorter total time. Similar trends hold across the other workloads (Appendix D [4]). Overall, although EvoTune incurs higher algorithm overhead than ML-based tuners, it is offset by a substantially lower replay cost, which dominates the total tuning time, as EvoTune requires far fewer query executions to reach high-quality configurations.

6.3 Evaluation on Space Localization

6.3.1 Ablation on Collaborative Detection. Figure 6 compares the tuning performance of different variants for tuning-space localization: (1) *Prior-Guided Selection*, which adaptively chooses among the lightweight model, the Detection Agent, and random exploration; (2) *Agent Only*, which relies solely on the Detection Agent; (3) *Small Model Only*, which uses only the lightweight model; (4) *Random*

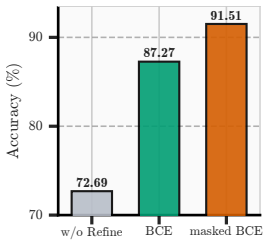


Figure 8: Refine Variants.

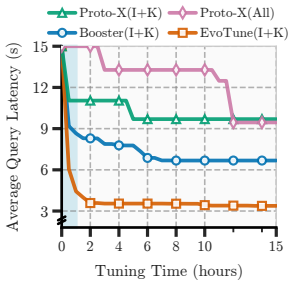


Figure 9: Tuning Analysis.

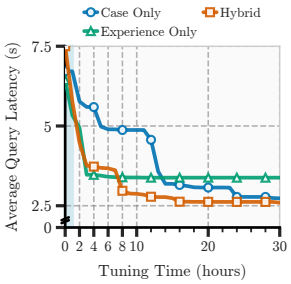


Figure 10: Memory Effect.

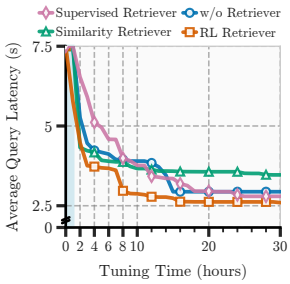


Figure 11: Retriever Analysis.

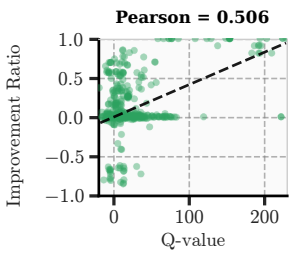


Figure 12: RL Retrieve Scatter.

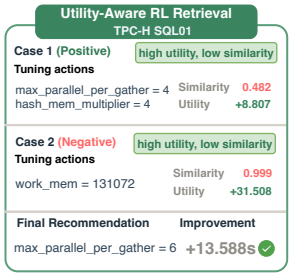


Figure 13: Retrieval Example.

Only, which relies solely on random exploration; (5) *No Random*, which disables random fallback; (6) *No Detection*, which directly recommends tuning actions without explicit tuning-space localization. The results show that *No Detection* suffers from sluggish convergence and easily gets trapped in local optima due to unguided exploration. Both *Agent Only* and *Small Model Only* improve over *No Detection*, but each remains limited by its own inductive bias: *Agent Only* captures query semantics and DBA knowledge, yet may overlook implicit execution regularities, while *Small Model Only* gradually learns recurring patterns from feedback but lacks the semantic reasoning needed for robust localization. *No Random* achieves rapid initial improvement, but soon stagnates at a local optimum because the search becomes overly exploitative. By adaptively balancing efficient pattern-based scoring, knowledge-driven reasoning, and random fallback, *Prior-Guided Selection* consistently achieves both faster convergence and better final performance.

6.3.2 Evaluation on Continual Refinement. We evaluate the design of continual refinement of the detection model from two perspectives: tuning effectiveness and localization accuracy. As shown in Figure 7, enabling *Refinement* consistently reduces average query latency and reaches strong performance earlier than *w/o Refinement*, indicating that refinement improves subspace localization throughout tuning. For localization accuracy, we evaluate accuracy on the target workload using ground-truth labels obtained with a sufficiently large tuning budget. To study the impact of different refinement objectives, we also compare *BCE* and *masked BCE*. As shown in Figure 8, both *BCE* and *masked BCE* improve over *w/o Refine*, and *masked BCE* achieves the highest accuracy of 91.51%. This confirms that masking unreliable negative feedback is critical for online update, as it reduces false-negative bias and yields more accurate component localization.

6.4 Evaluation on Action Recommendation

6.4.1 Comparison of Tuning Policies within the Same Subspace. Having shown the importance of subspace localization, we isolate tuning policy effects by comparing methods within the same subspace. We evaluate tuning efficiency on TPC-H SF10 using 7 queries with index- and knob-related anomalies, comparing the top two baselines (Proto-X and Booster) within the same I+K subspace, with *Proto-X(All)* as a full-space baseline. As shown in Figure 9, restricting Proto-X to the I+K subspace improves convergence over *Proto-X(All)*, highlighting the benefit of search space reduction. *EvoTune* achieves the best performance by more effectively exploiting observations and experiences, reaching strong configurations within the first few tuning rounds.

6.4.2 Evaluation on Memory Construction. EvoTune constructs a hybrid memory hub that integrates both fine-grained cases and generalized experience. To validate this design, we compare three variants: (1) *Case Only*, which uses only fine-grained cases; (2) *Experience Only*, which relies solely on generalized experience; and (3) *Hybrid*, our design that integrates both. As shown in Figure 10, the hybrid memory outperforms both *Experience Only* and *Case Only*, as it combines high-level guidance from coarse-grained experiences with fine-grained signals from individual cases.

6.4.3 Retrieval policy comparison. We compare four variants of the retriever module: (1) *w/o Retriever*, which performs zero-shot tuning without leveraging historical observations; (2) *Similarity Retriever*, which retrieves observations based on cosine similarity between queries; (3) *Supervised Retriever*, which ranks observations by predicting their utility given the current tuning context; and (4) *RL Retriever*, our approach that learns a utility-aware policy to prioritize observations with higher long-term impact. As shown in Figure 11, removing the retriever forces the Recommendation Agent to rely on unguided search, leading to inferior performance. The *Similarity Retriever* partially alleviates the cold-start issue. However, as tuning progresses, it repeatedly retrieves the same observations that provide limited or even misleading utility, causing the similarity-based approach to converge prematurely. The *Supervised Retriever* further improves by learning from data, but it optimizes for immediate utility and does not account for long-term effects through state transitions, limiting its ability to capture sequential

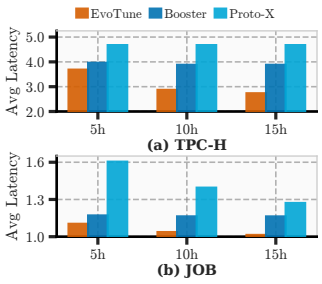


Figure 14: Initialization from Existing Tuner.

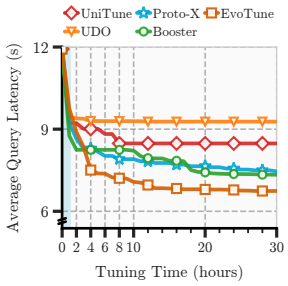


Figure 15: Cold Start on TiDB.

dependencies. In contrast, the *RL Retriever* learns to prioritize observations with higher downstream utility. As illustrated in Figure 12, the learned Q-values exhibit a stronger correlation with the final improvement ratio than similarity (Figure 2(c)). Although the overall correlation is moderate—particularly for low-Q regions—the observations with higher Q-values are consistently associated with positive performance gains, indicating that the retriever can effectively identify high-utility observations. Figure 13 further provides a concrete example: the RL retriever recalls a high-utility case despite low cosine similarity, enabling the agent to synthesize a better recommendation and achieve substantial improvement.

6.5 More Scenarios

6.5.1 Initialization from Existing Tuner. We further evaluate a warm-start setting where tuning begins from a pre-populated memory derived from an existing tuner. This setting is motivated by Booster, which is designed to assist existing tuners by leveraging pre-collected observations. To ensure a fair comparison, we initialize all methods from a memory hub constructed by running Proto-X for 12 hours, following Booster's original setup. Figure 14 reports the results. Recall that when starting from an empty memory (Section 6.2), Booster performs worse than Proto-X on JOB workload due to its limited exploration capability. In contrast, under this warm-start setting, Booster benefits from the initial exploration provided by Proto-X and outperforms it. EvoTune consistently achieves the best performance, benefiting from its ability to continuously evolve tuning policies through utility-aware retrieval and memory updates.

6.5.2 Robustness of Evolution under Cold Start (TiDB). To evaluate whether EvoTune can bootstrap memory-aware evolution on a previously unseen engine, we consider a cold-start setting where both the detection model and the retriever are initialized without offline pre-training, and the memory hub is empty at the start. We conduct experiments on TPC-H SF10 using a TiDB v8.5.3 cluster with 1 TiDB, 1 PD, 1 TiFlash, and 3 TiKV nodes. The TiDB tuning space includes 19 query-level knobs and 11 optimizer hints, while the index and query rewrite spaces follow the initialization strategy in Section 6.1. As shown in Figure 15, EvoTune underperforms in the first three hours due to the cold-start setting, where both the detection model and the retriever are initialized without prior training, limiting retrieval quality and localization accuracy. As tuning proceeds, newly collected observations are written back to memory and reused, enabling improved localization and more effective optimization. Consequently, EvoTune surpasses all baselines from the fourth hour onward. These results show that EvoTune can bootstrap effective memory online and steadily improve even without offline engine-specific experience.

7 Related Work

Anomaly Detection. Database anomaly detection seeks to identify the root causes of query performance degradation. Data-driven methods [22, 25, 42, 54] use system metrics, query plans, or execution statistics for diagnosis, but often generalize poorly under workload shifts due to their reliance on labeled data. Knowledge-driven methods [11, 29, 56] leverage LLMs to improve diagnostic reasoning, yet mainly depend on static prior knowledge and fail to utilize feedback signals for continual refinement. Besides, some single-component tuning approaches reduces the search space within individual components (e.g., knob selection [15]) but fails to capture component importance and interactions. In contrast, we identify anomaly-related components to localize a query-specific anomaly subspace.

LLM-Empowered DBMS Tuning. Recent studies have explored LLM-based agents for DBMS tuning across different tasks, including knob tuning [8, 14, 17, 20], query rewriting [21, 31], and index tuning [50]. However, these approaches typically focus on *single-component* optimization. Methods such as [8, 17, 31] encode manual knowledge into prompts, providing useful but largely static guidance. Other approaches [14, 20, 21, 50] incorporate historical feedback, but exploit it only in limited ways. Specifically, E2ETune [14] improves tuning ability via offline fine-tuning on historical observations, but incurs high data preparation cost and generalizes poorly under workload shifts. AgentTune [20] reduces this overhead by leveraging online feedback, but considers only top-k observations for the current query. LLMIdxAdvis [50] and LLM-R2 [21] rely on static similarity-based retrieval, which surface low-utility cases and lack mechanisms for continual adaptation. To the best of our knowledge, Booster [46] is the only LLM-based method that considers *multi-component* tuning. It retrieves query-level historical contexts and suggests per-query recommendations. However, it explores the entire configuration space without effective localization and still relies on similarity-based retrieval, making it prone to inefficient exploration and low-utility guidance.

8 Conclusion

Existing multi-component DBMS tuning approaches often suffer from unguided exploration and high iterative overhead. To address these limitations, we propose EvoTune, an LLM-empowered framework for autonomous multi-component DBMS tuning. EvoTune localizes high-impact tuning subspaces via anomaly detection and refines tuning policies through utility-aware retrieval. Moreover, it introduces a memory-aware mechanism that continuously exploits accumulated experience to jointly improve anomaly diagnosis and tuning decisions without fine-tuning the LLMs. Extensive experiments show that EvoTune consistently outperforms state-of-the-art baselines, achieving up to 44.5% performance improvement under the same tuning budget while reaching the best competing baseline performance with up to 3.9× less tuning time.

References

[1] 2015. TPC-DS Benchmark. https://www.tpc.org/tpcds/.

[2] 2015. TPC-H Benchmark. http://www.tpc.org/tpch/.

[3] 2025. pg_hint_plan. https://github.com/ossc-db/pg_hint_plan.

[4] 2026. EvoTune Appendix. https://anonymous.4open.science/r/DREAM-0A30/appendix.pdf.

[5] Christoph Anneser, Nesime Tatbul, David Cohen, Zhenggang Xu, Prithviraj Pandian, Nikolay Laptev, and Ryan Marcus. 2023. AutoSteer: Learned Query Optimization for Any SQL Database. *Proc. VLDB Endow.* 16, 12 (Aug. 2023), 3515–3527. doi:10.14778/3611540.3611544

[6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 159, 25 pages.

[7] Sébastien Bubeck and Nicolò Cesa-Bianchi. 2012. Regret Analysis of Stochastic and Nonstochastic Multi-armed Bandit Problems. *Found. Trends Mach. Learn.* 5, 1 (2012), 1–122. doi:10.1561/2200000024

[8] Sibei Chen, Ju Fan, Bin Wu, Nan Tang, Chao Deng, Pengyi Wang, Ye Li, Jian Tan, Feifei Li, Jingren Zhou, and Xiaoyong Du. 2025. Automatic Database Configuration Debugging using Retrieval-Augmented Language Models. *Proc. ACM Manag. Data* 3, 1, Article 13 (Feb. 2025), 27 pages.

[9] Florin Cuconasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, Cesare Campagnano, Yoelle Maarek, Nicola Tonellotto, and Fabrizio Silvestri. 2024. The Power of Noise: Redefining Retrieval for RAG Systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Washington DC, USA) *(SIGIR '24)*. Association for Computing Machinery, New York, NY, USA, 719–729. doi:10.1145/3626772.3657834

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, 4171–4186.

[11] Victor Giannakouris and Immanuel Trummer. 2024. DBG-PT: A Large Language Model Assisted Query Performance Regression Debugger. *Proc. VLDB Endow.* 17, 12 (Aug. 2024), 4337–4340.

[12] Victor Giannakouris and Immanuel Trummer. 2025. λ-Tune: Harnessing Large Language Models for Automated Database System Tuning. *Proc. ACM Manag. Data* 3, 1, Article 2 (Feb. 2025), 26 pages.

[13] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. 2019. Soft Actor-Critic Algorithms and Applications. doi:10.48550/arXiv.1812.05905 arXiv:1812.05905 [cs].

[14] Xinmei Huang, Haoyang Li, Jing Zhang, Xinxin Zhao, Zhiming Yao, Yiyan Li, Tieying Zhang, Jianjun Chen, Hong Chen, and Cuiping Li. 2024. E2ETune: End-to-End Knob Tuning via Fine-tuned Generative Language Model. *CoRR* abs/2404.11581 (2024).

[15] Konstantinos Kanellis, Ramnatthan Alagappan, and Shivaram Venkataraman. 2020. Too Many Knobs to Tune? Towards Faster Database Tuning by Pre-selecting Important Knobs. In *HotStorage*. USENIX Association.

[16] Ron Kohavi, Randal M. Henne, and Dan Sommerfield. 2007. Practical guide to controlled experiments on the web: listen to your customers not to the hippo. In *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Jose, California, USA) *(KDD '07)*. Association for Computing Machinery, New York, NY, USA, 959–967.

[17] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2025. GPTuner: An LLM-Based Database Tuning System. *SIGMOD Rec.* 54, 1 (April 2025), 101–110.

[18] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter A. Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proc. VLDB Endow.* 9, 3 (2015), 204–215.

[19] Xinzhe Li. 2025. A Review of Prominent Paradigms for LLM-Based Agents: Tool Use, Planning (Including RAG), and Feedback Learning. In *Proceedings of the 31st International Conference on Computational Linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, 9760–9779.

[20] Yiyan Li, Haoyang Li, Jing Zhang, Renata Borovica-Gajic, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, Cuiping Li, and Hong Chen. 2025. AgentTune: An Agent-Based Large Language Model Framework for Database Knob Tuning. *Proc. ACM Manag. Data* 3, 6, Article 293 (Dec. 2025), 29 pages.

[21] Zhaodonghui Li, Haitao Yuan, Huiming Wang, Gao Cong, and Lidong Bing. 2024. LLM-R2: A Large Language Model Enhanced Rule-Based Rewrite System for Boosting Query Efficiency. *Proc. VLDB Endow.* 18, 1 (Sept. 2024), 53–65.

[22] Minghua Ma, Zheng Yin, Shenglin Zhang, Sheng Wang, Christopher Zheng, Xinhao Jiang, Hanwen Hu, Cheng Luo, Yilin Li, Nengjun Qiu, Feifei Li, Changcheng Chen, and Dan Pei. 2020. Diagnosing Root Causes of Intermittent Slow Queries in Large-Scale Cloud Databases. *Proc. VLDB Endow.* 13, 8 (2020), 1176–1189.

[23] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) *(SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 1275–1288.

[24] Leland McInnes, John Healy, and Steve Astels. 2017. HDBSCAN: Hierarchical density based clustering. *Journal of Open Source Software* 2, 11 (2017), 205. doi:10.21105/joss.00205

[25] Biao Ouyang, Yingying Zhang, Hanyin Cheng, Yang Shu, Chenjuan Guo, Bin Yang, Qingsong Wen, Lunting Fan, and Christian S. Jensen. 2024. RCRank: Multimodal Ranking of Root Causes of Slow Queries in Cloud Database Systems. *Proc. VLDB Endow.* 18, 4 (2024), 1169–1182.

[26] Yu Shen, Beicheng Xu, Yupeng Lu, Donghui Chen, Huaijun Jiang, Zhipeng Xie, Senbo Fu, Nan Zhang, Yuxin Ren, Ning Jia, Xinwei Hu, and Bin Cui. 2025. A-Tune-Online: Efficient and QoS-Aware Online Configuration Tuning for Dynamic Workloads. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 2161–2173. doi:10.1109/ICDE65448.2025.00164

[27] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 377, 19 pages.

[28] Tarique Siddiqui and Wentao Wu. 2024. ML-Powered Index Tuning: An Overview of Recent Progress and Open Challenges. *SIGMOD Rec.* 52, 4 (Jan. 2024), 19–30.

[29] Vikramank Y. Singh, Kapil Vaidya, Vinayshekhar Bannihatti Kumar, Sopan Khosla, Balakrishnan Narayanaswamy, Rashmi Gangadharaiah, and Tim Kraska. 2024. Panda: Performance Debugging for Databases using LLM Agents. In *14th Conference on Innovative Data Systems Research, CIDR 2024, Chaminade, HI, USA, January 14-17, 2024*. www.cidrdb.org.

[30] Zhaoyan Sun. 2025. R-Bot: An LLM-based Query Rewrite System. https://github.com/curtis-sun/LLM4Rewrite/blob/main/explain_rule/calcite_rewrite_rules_structured.jsonl Accessed: Dec. 26, 2025.

[31] Zhaoyan Sun, Xuanhe Zhou, Guoliang Li, Xiang Yu, Jianhua Feng, and Yong Zhang. 2025. R-Bot: An LLM-based Query Rewrite System. *Proc. VLDB Endow.* 18, 12 (2025), 5031–5044.

[32] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) *(SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 1009–1024.

[33] Junxiong Wang, Immanuel Trummer, and Debabrota Basu. 2021. UDO: Universal Database Optimization using Reinforcement Learning. *Proc. VLDB Endow.* 14, 13 (2021), 3402–3414.

[34] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. 2024. A survey on large language model based autonomous agents. *Frontiers Comput. Sci.* 18, 6 (2024), 186345.

[35] Shuxian Wang, Sicheng Pan, and Alvin Cheung. 2024. QED: A Powerful Query Equivalence Decider for SQL. *Proc. VLDB Endow.* 17, 11 (July 2024), 3602–3614.

[36] Xiaoying Wang, Wentao Wu, Vivek Narasayya, and Surajit Chaudhuri. 2025. Esc: An Early-Stopping Checker for Budget-Aware Index Tuning. *Proc. VLDB Endow.* 18, 5 (Aug. 2025), 1278–1290.

[37] Zijia Wang, Haoran Liu, Chen Lin, Zhifeng Bao, Guoliang Li, and Tianqing Wang. 2024. Leveraging Dynamic and Heterogeneous Workload Knowledge to Boost the Performance of Index Advisors. *Proc. VLDB Endow.* 17, 7 (March 2024), 1642–1654.

[38] Lucas Woltmann, Jerome Thiessat, Claudio Hartmann, Dirk Habich, and Wolfgang Lehner. 2023. FASTgres: Making Learned Query Optimizer Hinting Effective. *Proc. VLDB Endow.* 16, 11 (July 2023), 3310–3322.

[39] Xiaokun Wu, Min Chen, Wanyi Li, Rui Wang, Limeng Lu, Jia Liu, Kai Hwang, Yixue Hao, Yanru Pan, Qingguo Meng, Kaibin Huang, Long Hu, Mohsen Guizani, Naipeng Chao, Giancarlo Fortino, Fei Lin, Yonglin Tian, Dusit Niyato, and Fei-Yue Wang. 2025. LLM Fine-Tuning: Concepts, Opportunities, and Challenges. *Big Data Cogn. Comput.* 9, 4 (2025), 87.

[40] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, Qi Zhang, and Tao Gui. 2025. The rise and potential of large language model based agents: a survey. *Sci. China Inf. Sci.* 68, 2 (2025).

[41] Xianghong Xu, Zhibing Zhao, Tieying Zhang, Rong Kang, Luming Sun, and Jianjun Chen. 2023. COOOL: A Learning-To-Rank Approach for SQL Hint Recommendations. In *Joint Proceedings of Workshops at the 49th International Conference on Very Large Data Bases (VLDB 2023), Vancouver, Canada, August 28 - September 1, 2023.*

[42] Dong Young Yoon, Ning Niu, and Barzan Mozafari. 2016. DBSherlock: A Performance Diagnostic Tool for Transactional Databases. In *Proceedings of the 2016 International Conference on Management of Data* (San Francisco, California, USA) *(SIGMOD '16)*. Association for Computing Machinery, New York, NY, USA, 1599–1614.

[43] Tao Yu, Zhaonian Zou, Weihua Sun, and Yu Yan. 2024. Refactoring Index Tuning Process with Benefit Estimation. *Proc. VLDB Endow.* 17, 7 (March 2024), 1528–1541.

[44] Ji Zhang, Yu Liu, Ke Zhou, Guoliang Li, Zhili Xiao, Bin Cheng, Jiashu Xing, Yangtao Wang, Tianheng Cheng, Li Liu, Minwei Ran, and Zekang Li. 2019. An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning. In *Proceedings of the 2019 International Conference on Management of Data*. Association for Computing Machinery, 415–432.

[45] William Zhang, Wan Shen Lim, Matthew Butrovich, and Andrew Pavlo. 2024. The Holon Approach for Simultaneously Tuning Multiple Components in a Self-Driving Database Management System with Machine Learning via Synthesized Proto-Actions. *Proc. VLDB Endow.* 17, 11 (2024), 3373–3387.

[46] William Zhang, Wan Shen Lim, and Andrew Pavlo. 2026. This is Going to Sound Crazy, But What If We Used Large Language Models to Boost Automatic Database Tuning Algorithms By Leveraging Prior History? We Will Find Better Configurations More Quickly Than Retraining From Scratch! *Proc. ACM Manag. Data* 4, 1, Article 90 (April 2026), 29 pages. doi:10.1145/3786704

[47] Xinyi Zhang, Zhuo Chang, Hong Wu, Yang Li, Jia Chen, Jian Tan, Feifei Li, and Bin Cui. 2023. A Unified and Efficient Coordinating Framework for Autonomous DBMS Tuning. *Proc. ACM Manag. Data* 1, 2, Article 186 (June 2023), 26 pages.

[48] Xinyi Zhang, Hong Wu, Zhuo Chang, Shuowei Jin, Jian Tan, Feifei Li, Tieying Zhang, and Bin Cui. 2021. ResTune: Resource Oriented Tuning Boosted by Meta-Learning for Cloud Databases. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) *(SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2102–2114.

[49] Xinyi Zhang, Hong Wu, Yang Li, Jian Tan, Feifei Li, and Bin Cui. 2022. Towards Dynamic and Safe Configuration Tuning for Cloud Databases. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 631–645. doi:10.1145/3514221.3526176

[50] Xinxin Zhao, Haoyang Li, Jing Zhang, Xinmei Huang, Tieying Zhang, Jianjun Chen, Rui Shi, Cuiping Li, and Hong Chen. 2025. LLMIdxAdvis: Resource-Efficient Index Advisor Utilizing Large Language Model. *CoRR* abs/2503.07884 (2025). arXiv:2503.07884

[51] Yue Zhao, Gao Cong, Jiachen Shi, and Chunyan Miao. 2022. QueryFormer: A Tree Transformer Model for Query Plan Representation. *Proc. VLDB Endow.* 15, 8 (2022), 1658–1670.

[52] Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, and Jun Wang. 2025. Memento: Fine-tuning LLM Agents without Fine-tuning LLMs. *CoRR* abs/2508.16153 (2025).

[53] Wei Zhou, Chen Lin, Xuanhe Zhou, and Guoliang Li. 2024. Breaking It Down: An In-Depth Study of Index Advisors. *Proc. VLDB Endow.* 17, 10 (June 2024), 2405–2418.

[54] Xuanhe Zhou, Lianyuan Jin, Ji Sun, Xinyang Zhao, Xiang Yu, Jianhua Feng, Shifu Li, Tianqing Wang, Kun Li, and Luyang Liu. 2021. DBMind: a self-driving platform in openGauss. *Proc. VLDB Endow.* 14, 12 (July 2021), 2743–2746.

[55] Xuanhe Zhou, Guoliang Li, Chengliang Chai, and Jianhua Feng. 2021. A learned query rewrite system using Monte Carlo tree search. *Proc. VLDB Endow.* 15, 1 (Sept. 2021), 46–58.

[56] Xuanhe Zhou, Guoliang Li, Zhaoyan Sun, Zhiyuan Liu, Weize Chen, Jianming Wu, Jiesi Liu, Ruohang Feng, and Guoyang Zeng. 2024. D-Bot: Database Diagnosis System using Large Language Models. *Proc. VLDB Endow.* 17, 10 (2024), 2514–2527.

[57] Rong Zhu, Wei Chen, Bolin Ding, Xingguang Chen, Andreas Pfadler, Ziniu Wu, and Jingren Zhou. 2023. Lero: A Learning-to-Rank Query Optimizer. *Proc. VLDB Endow.* 16, 6 (Feb. 2023), 1466–1479.